



The Class NP (7.3)

COT 4210 Discrete Structures II
Summer 2025
Department of Computer Science
Dr. Steinberg

A Harder Problem

Hamiltonian Path

- A **Hamiltonian path** in a directed graph is a directed path that goes through **each node** exactly **once**. The related language is obvious enough:

$$HAMPATH = \{\langle G, s, t \rangle \mid \text{A Hamiltonian path exists from } s \text{ to } t \text{ in } G\}$$

- We can find an exponential-time algorithm easily, based off our brute-force algorithm for *PATH*: just modify the sequence checker to check whether the path is Hamiltonian.
- What we don't know is any way to check for a Hamiltonian path in polynomial time. In fact, it turns out that we don't know that in a really important way.
- The Hamiltonian path problem, however, has another interesting characteristic: We know a much easier way to *check* a solution than to *find* one.

Polynomial Verifiability

$HAMPATH = \{\langle G, s, t \rangle \mid \text{A Hamiltonian path exists from } s \text{ to } t \text{ in } G\}$

- Consider the problem of, *given* an alleged Hamiltonian path, verifying that it *is* one. The required algorithm has the following steps:
 - Check that the path goes through every node in G .
 - Check that the path doesn't repeat any nodes.
- This is obviously doable, and obviously doable polynomially.
 - (It isn't always.)
- This kind of algorithm is called a **verifier**.

Definition: Verifier

A **verifier** for a language A is an algorithm V so that:

$$A = \{w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}$$

c is called the **certificate** of w 's membership in A .

- In the case of *HAMPATH* the certificate is simply the actual solution.

Now let's bound this by complexity:

- A verifier is a **polynomial time verifier** if it runs in polynomial time with respect to the length of w .
- A language A is **polynomially verifiable** if it has a polynomial time verifier.

This is going somewhere, but let's pause for a moment and look at *HAMPATH* from another perspective.

Nondeterministic *HAMPATH*

M = On input $\langle G, s, t \rangle$:

1. Create a sequence x of n nodes in G as follows:
 - i. Let $x_1 = s$.
 - ii. Let $x_n = t$.
 - iii. Let $x_2 \dots x_{n-1}$ be nondeterministically chosen from the nodes in G other than s and t .
2. If there are any repetitions in x , reject.
3. For $1 \leq i < n$, if (x_i, x_{i+1}) is not an edge in G , reject.
4. If we have not rejected, accept.

Nondeterministic *HAMPATH*

M = On input $\langle G, s, t \rangle$:

1. Create a sequence x of n nodes in G as follows:
 - i. Let $x_1 = s$.
 - ii. Let $x_n = t$.
 - iii. Let $x_2 \dots x_{n-1}$ be nondeterministically chosen from the nodes in G other than s and t .
2. If there are any repetitions in x , reject.
3. For $1 \leq i < n$, if (x_i, x_{i+1}) is not an edge in G , reject.
4. If we have not rejected, accept.

Here we've taken a nondeterministic Turing machine and thrown it at Hamiltonian path.

It's easy to see that, other than the nondeterministic selection itself, every part of this algorithm runs in polynomial time.

We say, therefore, that this algorithm runs in **nondeterministic polynomial time**.

And, in fact, we can say a lot more than that.

Notice the form of this algorithm:

- Nondeterministically guess a solution.
- Polynomially verify that solution.

Nondeterministic *HAMPATH*

M = On input $\langle G, s, t \rangle$:

1. Create a sequence x of n nodes in G as follows:
 - i. Let $x_1 = s$.
 - ii. Let $x_n = t$.
 - iii. Let $x_2 \dots x_{n-1}$ be nondeterministically chosen from the nodes in G other than s and t .
2. If there are any repetitions in x , reject.
3. For $1 \leq i < n$, if (x_i, x_{i+1}) is not an edge in G , reject.
4. If we have not rejected, accept.

Here we've taken a nondeterministic Turing machine and thrown it at Hamiltonian path.

It's easy to see that, other than the nondeterministic selection itself, every part of the algorithm runs in polynomial time.

This form generalizes.

The algorithm runs in **nondeterministic polynomial time**.

And, in fact, we can say a lot more than that.

Notice the form of this algorithm:

- Nondeterministically guess a solution.
- Polynomially verify that solution.

The Class NP

Verifiers and Nondeterminism

Assume that a machine V exists so that V :

- Is a deterministic Turing machine,
- Runs in polynomial time n^k , and
- Is a verifier for language A .

Then we can construct a nondeterministic Turing machine that decides A in polynomial time, as follows:

M = On input w of length n :

1. Nondeterministically select string c , of length at most n^k .
2. Run V on $\langle w, c \rangle$.
3. If V accepts, accept; reject otherwise.

Assume that a machine M exists so that M :

- Is a nondeterministic Turing machine,
- Runs in ND-polynomial time n^k , and
- Is a decider for language A .

Then we can construct a deterministic Turing machine that verifies A in polynomial time, as follows:

V = On input $\langle w, c \rangle$:

1. Simulate M on w , using c to choose each branch of nondeterminism.
2. If this branch of M accepts, accept; otherwise reject.

Verifiers and Nondeterminism

Assume that a machine

- Is a deterministic Turing machine,
- Runs in polynomial time n^k , and
- Is a verifier for language A .

Then we can construct a machine that decides A as follows:

$M =$ On input w of length n :

1. Nondeterministically choose each c to choose each c^k .
2. Run V on $\langle w, c \rangle$.
3. If V accepts, accept; reject otherwise.

The class of languages with deterministic polynomial-time verifiers is equivalent to the class of languages decidable by nondeterministic polynomial-time Turing machines.

M exists so that M :

Is a deterministic Turing machine, runs in polynomial time n^k , and is a verifier for language A .

M is a deterministic Turing machine that runs in polynomial time, as follows:

using c to choose each c^k .
minimism.

2. If this branch of M accepts, accept; otherwise reject.

The Class NP

- First, let's get our formal complexity class notation for nondeterministic Turing machines out of the way:

$$\mathbf{NTIME}(t(n)) = \{ L \mid L \text{ is a language decided by an } O(t(n))\text{-time NDTM} \}$$

- And with that, we can define the class NP.

$$\mathbf{NP} = \bigcup_k [\mathbf{NTIME}(n^k)] = \{ A \mid A \text{ is a language with a polynomial time verifier} \}$$

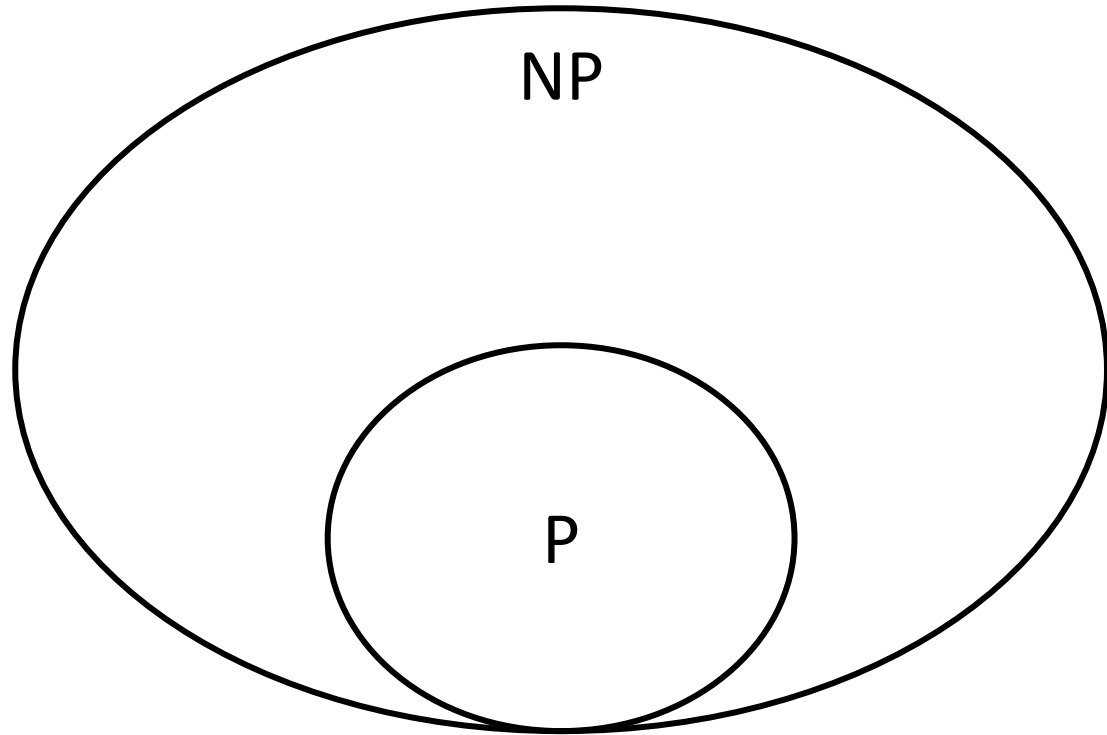
- Like P, NP is robust across computational models given that those models are polynomially equivalent. This means that we can analyze nondeterministic problems, generally, the same way we can deterministic ones.

P and NP

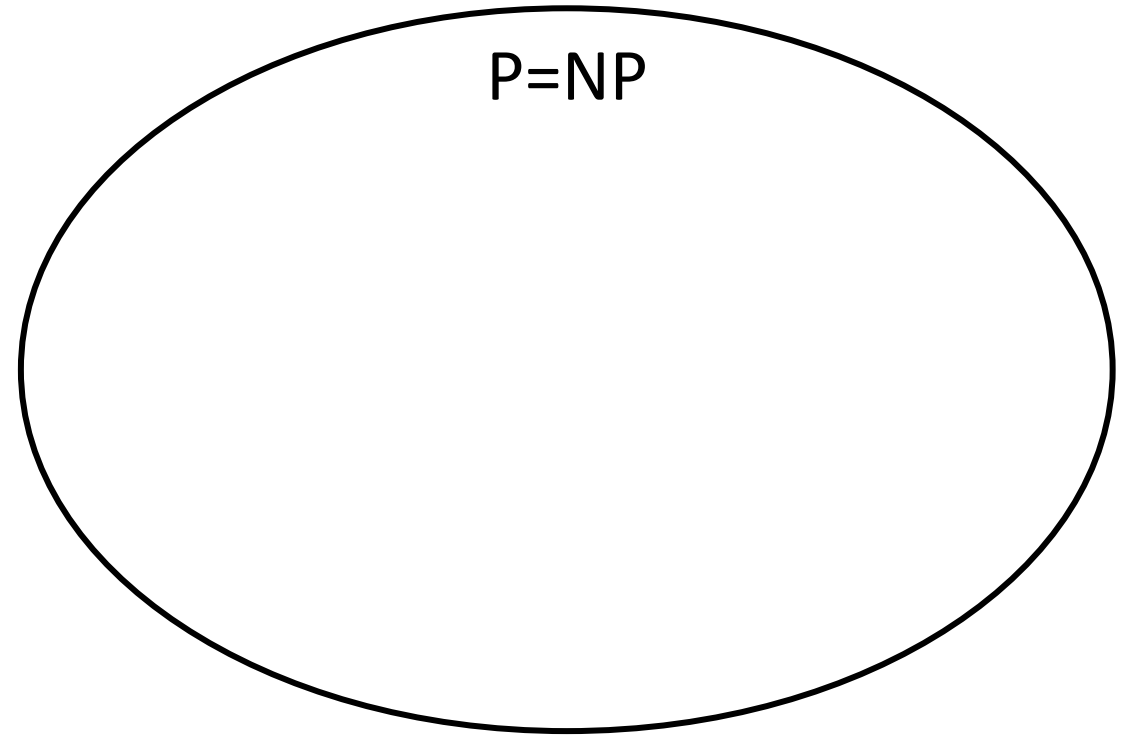
- We know that $P \subseteq NP$ simply because a non-deterministic Turing machine can do anything a deterministic one can. However, this leads to one of the more remarkable unsolved problems in the sciences.

We do not know whether or not $NP \subseteq P$.

- No one has yet managed to prove either that there is or is not a way to overcome this requirement for nondeterminism.
 - It *seems* intuitive that verification is easier than decision, but that intuition has yet to lead to a proof.
 - Conversely, enormous effort has been expended trying to prove that $NP = P$, without result.
- The best **guess** of most researchers at this point is that $NP \neq P$, because of the amount of effort that has been spent trying to prove the opposite. But we don't know – and given the implications of Gödel's incompleteness theorem, we may never.



Option 1



Option 2

Some NP Reductions

Review: The Classes P and NP

P is the class of languages that are decidable in polynomial time by a deterministic Turing machine.

$$P = \bigcup_k [\text{TIME}(n^k)]$$

P and NP are both robust across reasonable computational models.

NP is the class of languages that are decidable in polynomial time by a nondeterministic Turing machine, or, equally, the class of languages that can be verified in polynomial time by a deterministic Turing machine.

$$NP = \bigcup_k [\text{NTIME}(n^k)]$$

$NP = \{ A \mid A \text{ is a language with a polynomial-time verifier} \}$

Clique

- A **clique** in an undirected graph is a completely connected subgraph – that is, a subgraph where every two nodes are connected by an edge. A ***k*-clique** is a clique of *k* nodes.
- The clique problem is to determine whether a graph contains a clique of a specified size. In language form:
- $CLIQUE = \{ \langle G, k \rangle \mid G \text{ is an undirected graph with a } k\text{-clique} \}$
- $CLIQUE \in NP$. Let's prove it.

Clique

- A **clique** in an undirected graph is a completely connected subgraph – that is, a subgraph where every two nodes are connected by an edge. A **k -clique** is a clique of k nodes.
- The clique problem is to determine whether a graph contains a clique of a specified size. In language form:
- $CLIQUE = \{ \langle G, k \rangle \mid G \text{ is an undirected graph with a } k\text{-clique} \}$
- $CLIQUE \in NP$. Let's prove it.
- $V = \text{On input } \langle \langle G, k \rangle, c \rangle$:
 1. Test if c is a subgraph of G . If not, reject.
 2. Test if c has k nodes. If not, reject.
 3. For every pair of nodes (a, b) in c :
 - a. If (a, b) is not an edge in G , reject.
 4. If we have not rejected, accept.
- $M = \text{On input } \langle G, k \rangle$:
 1. Nondeterministically select c to be a subgraph of G with k nodes.
 2. For every pair of nodes (a, b) in c :
 - a. If (a, b) is not an edge in G , reject.
 3. If we have not rejected, accept.

Sums of Subsets

- Given a bag of numbers $X = \{x_1, x_2, \dots, x_n\}$, we want to determine whether it has a sub-bag that adds up to some number t .

- $SUBSET-SUM =$

$$\{ \langle X, t \rangle \mid X = \{x_1, \dots, x_n\}, \text{ and for some } Y = \{y_1, \dots, y_n\}, Y \subseteq X, \sum y = t \}$$

- $SUBSET-SUM \in NP$. Let's prove it.

Sums of Subsets

- Given a bag of numbers $X = \{x_1, x_2, \dots, x_n\}$, we want to determine whether it has a sub-bag that adds up to some number t .

- $SUBSET-SUM =$

$$\{ \langle X, t \rangle \mid X = \{x_1, \dots, x_n\}, \text{ and for some } Y = \{y_1, \dots, y_n\}, Y \subseteq X, \sum y = t \}$$

- $SUBSET-SUM \in NP$. Let's prove it.

- $V =$ **On input** $\langle \langle X, t \rangle, C \rangle$:
 - Test if $C \subseteq X$. If not, reject.
 - Test if $\sum c = t$. If not, reject.
 - If we have not rejected, accept.
- $M =$ **On input** $\langle G, k \rangle$:
 - Nondeterministically select $C \subseteq X$.
 - Test if $\sum c = t$. If not, reject.
 - If we have not rejected, accept.

Satisfiability

- A **Boolean** formula is an expression involving Boolean variables and operations:

$$\phi = (\bar{x} \wedge y) \vee (x \wedge \bar{z})$$

- It is **satisfiable** if there is an assignment to its variables that makes it evaluate to true. (0, 1, 0) suffices for the above. (So do some other assignments; can you find them?)

$$SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula}\}$$

- The satisfiability problem has significant implications. For now, let's just prove it's in NP.

Satisfiability

- A **Boolean** formula is an expression involving Boolean variables and operations:

$$\phi = (\bar{x} \wedge y) \vee (x \wedge \bar{z})$$

- It is **satisfiable** if there is an assignment to its variables that makes it evaluate to true. (0, 1, 0) suffices for the above. (So do some other assignments; can you find them?)

$$SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula}\}$$

- The satisfiability problem has significant implications. For now, let's just prove it's in NP.

- $V =$ **On input** $\langle \phi, C \rangle$:
 1. Test if C is a valid assignment to ϕ 's variables. If not, reject.
 2. Test if C satisfies ϕ . If not, reject.
 3. If we have not rejected, accept.
- $M =$ **On input** $\langle \phi \rangle$:
 1. Nondeterministically select C , a set of assignments to ϕ 's variables.
 2. Test if C satisfies ϕ . If not, reject.
 3. If we have not rejected, accept.

3-Satisfiability

- A Boolean formula is in **conjunctive normal form**, or, equally, is a **cnf-formula**, if it is composed as follows:
 - The formula itself is a set of **clauses** connected by AND operations.
 - Each **clause** is a set of literals connected by OR operations.
 - Each **literal** is either a variable or its negation.

$\phi = (\bar{x} \wedge y) \vee (x \wedge \bar{z})$ is not in CNF, but $\phi = (\bar{x} \vee y) \wedge (x \vee \bar{z})$ is.

- Further, a Boolean formula is a **3cnf-formula** if it is a cnf-formula where all clauses have exactly three literals, such as:

$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee y \vee \bar{z})$$

- Given that, we can write:

$$3SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3cnf-formula}\}$$

3-Satisfiability

- A Boolean formula is:
 - **Satisfiable** if there is an assignment to its variables that makes it evaluate to true.
 - A **cnf-formula** if it is a set of literals OR'd into clauses in turn AND'd into the formula.
 - A **3cnf-formula** if it is a cnf-formula where all clauses have exactly three literals, such as:

$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee y \vee \bar{z})$$

- We want to determine whether a 3cnf-formula is satisfiable:

$$3SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3cnf-formula}\}$$

3-Satisfiability

- A Boolean formula is:
 - **Satisfiable** if there is an assignment to its variables that makes it evaluate to true.
 - A **cnf-formula** if it is a set of literals OR'd into clauses in turn AND'd into the formula.
 - A **3cnf-formula** if it is a cnf-formula where all clauses have exactly three literals, such as:

$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee y \vee \bar{z})$$

- We want to determine whether a 3cnf-formula is satisfiable:

$$3SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3cnf-formula}\}$$

- $V =$ **On input** $\langle \phi, C \rangle$:
 1. Test if C is a valid assignment to ϕ 's variables. If not, reject.
 2. Test if C satisfies ϕ . If not, reject.
 3. If we have not rejected, accept.
- $M =$ **On input** $\langle \phi \rangle$:
 1. Nondeterministically select C , a set of assignments to ϕ 's variables.
 2. Test if C satisfies ϕ . If not, reject.
 3. If we have not rejected, accept.

Review: Functions and Reduction

Definition: Computable Function

- A function $f: \Sigma^* \rightarrow \Sigma^*$ is a **computable function** if some Turing machine M , on every input s , halts with only $f(s)$ on its tape.

Definition: Mapping Reducibility

- To indicate that language A is **mapping reducible** to another language B , we write $A \leq_m B$.
- $A \leq_m B$ if there's a computable function $f: \Sigma^* \rightarrow \Sigma^*$ so that for every s ,

$$s \in A \iff f(s) \in B.$$

Polynomial Reducibility

Definition: Computable Function

- A function $f: \Sigma^* \rightarrow \Sigma^*$ is a **computable function** if some Turing machine M , on every input s , halts with only $f(s)$ on its tape.

Definition: Mapping Reducibility

- To indicate that language A is **mapping reducible** to another language B , we write $A \leq_m B$.
- $A \leq_m B$ if there's a computable function $f: \Sigma^* \rightarrow \Sigma^*$ so that for every s ,

$$s \in A \iff f(s) \in B.$$

We can add complexity in the obvious manner.

A **polynomial-time computable function** is a computable function so that the Turing machine M that computes it runs in polynomial time.

A language A is **polynomial-time mapping reducible**, or **polynomial-time reducible**, to another language B if $A \leq_m B$ and the function reducing them is polynomial-time computable. We write $A \leq_p B$.

Reducibility and its Implications

- Consider languages A and B , let M_B be a polynomial-time algorithm deciding B , and let f be a polynomial-time reduction from A to B . We can decide A in polynomial time as follows:

M_A = On input s :

1. Compute $f(s)$.
 2. Run M_B on $f(s)$.
 3. Output the output of M_B .
- **The class P is closed under polynomial reduction.**

If $A \leq_P B$ and $B \in P, A \in P$.

Reducing 3-Satisfiability to Clique

- Consider a 3cnf-formula with k clauses:
$$\phi = (x_1 \vee y_1 \vee z_1) \wedge (x_2 \vee y_2 \vee z_2) \wedge \cdots \wedge (x_k \vee y_k \vee z_k)$$
- We can reduce *3SAT* to *CLIQUE* by generating the string $\langle G, k \rangle$, with G a graph as follows:
 - The graph has $3k$ nodes.
 - The nodes are divided into **triplets** of 3 nodes each, corresponding to the clauses.
 - Each node in a triplet corresponds to a literal.
 - All nodes in G are directly connected, *except*:
 - Nodes in the same triplet are not connected.
 - Nodes corresponding to contradictory literals are not connected.

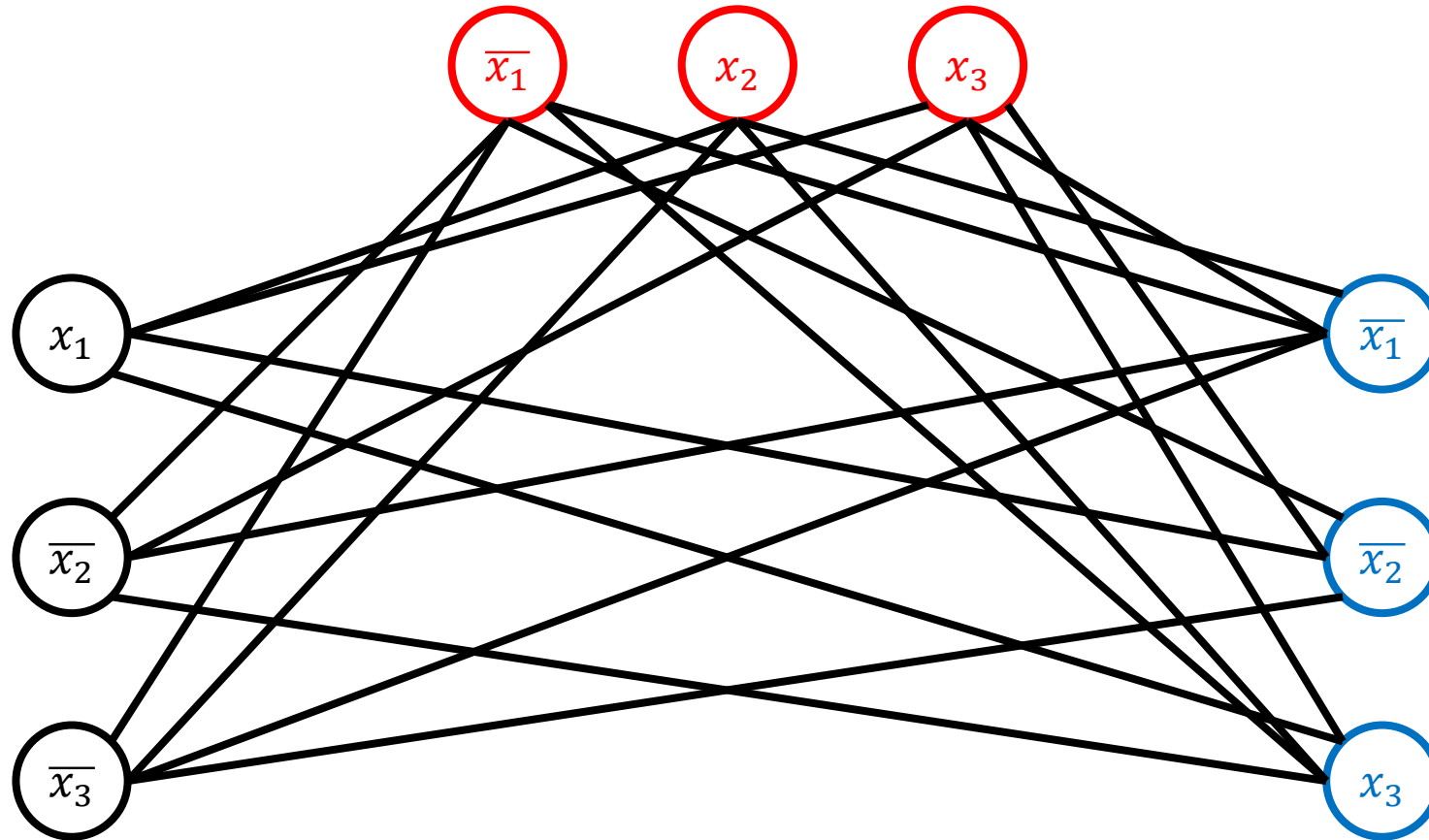
Reducing 3-Satisfiability to Clique

- Consider a 3cnf-formula with k clauses:
$$\phi = (x_1 \vee y_1 \vee z_1) \wedge (x_2 \vee y_2 \vee z_2) \wedge \cdots \wedge (x_k \vee y_k \vee z_k)$$
- We can reduce *3SAT* to *CLIQUE* by generating the string $\langle G, k \rangle$, with G a graph as follows:
 - The graph has $3k$ nodes.
 - The nodes are divided into **triplets** of 3 nodes each, corresponding to the clauses.
 - Each node in a triplet corresponds to a literal.
 - All nodes in G are directly connected, *except*:
 - Nodes in the same triplet are not connected.
 - Nodes corresponding to contradictory literals are not connected.
- Suppose that ϕ has a satisfying assignment.
 - Then in each of the k clauses, at least one literal is true.
 - For each clause C_i , select a true literal L_i .
- By the construction of G , we observe the following:
 - Each selected L_i has a corresponding selected node v_i .
 - v_i is in a different triplet from every other selected node, because we selected each literal from a different clause.
 - L_i cannot contradict any other selected literal, because they are all true.
 - v_i is connected to every other selected node, unless:
 - They are in the same triplet, which they are not.
 - They are contradictory, which they are not.
- All k nodes $v_1 \dots v_k$ are connected, hence a clique.

Reducing 3-Satisfiability to Clique

- Consider a 3cnf-formula with k clauses:
$$\phi = (x_1 \vee y_1 \vee z_1) \wedge (x_2 \vee y_2 \vee z_2) \wedge \cdots \wedge (x_k \vee y_k \vee z_k)$$
- We can reduce *3SAT* to *CLIQUE* by generating the string $\langle G, k \rangle$, with G a graph as follows:
 - The graph has $3k$ nodes.
 - The nodes are divided into **triplets** of 3 nodes each, corresponding to the clauses.
 - Each node in a triplet corresponds to a literal.
 - All nodes in G are directly connected, *except*:
 - Nodes in the same triplet are not connected.
 - Nodes corresponding to contradictory literals are not connected.
- Suppose that G has a k -clique C . Observe that:
 - No two of C 's nodes are in the same triple, because those nodes cannot be connected.
 - Therefore, each of the triples contains exactly one of C 's nodes.
 - None of C 's nodes correspond to contradictory literals.
- Assign truth values to the variables of ϕ so that each individual literal corresponding one of C 's nodes is made true.
 - Since each triple contains one of C 's nodes, each clause has a literal made true.
 - Since none of C 's nodes correspond to contradictory literals, this assignment must be possible.
- The assignment is possible, so ϕ is satisfiable.

$$\phi = (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$



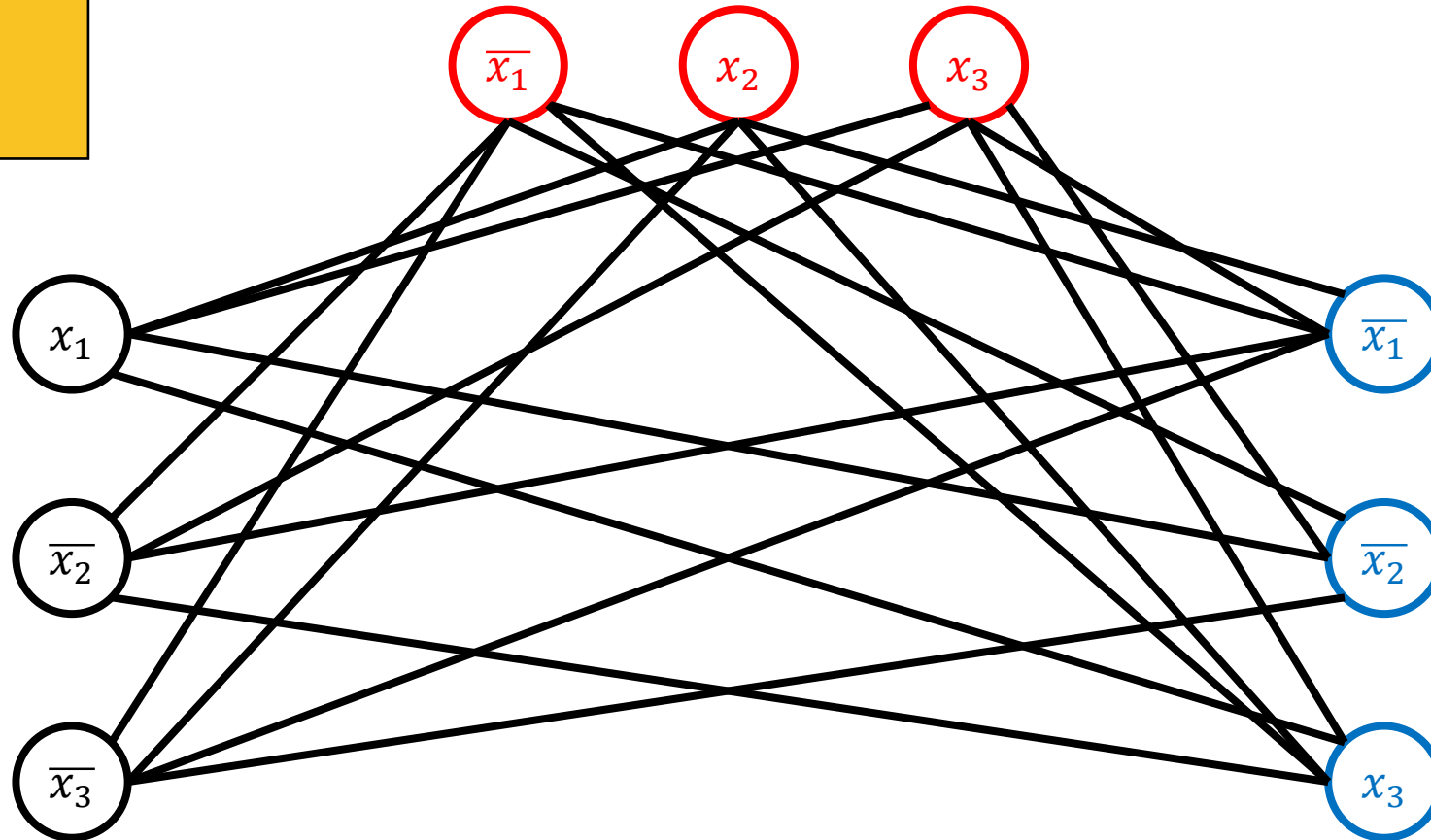
$$\phi = (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$

Consider the assignment:

$$x_1 = T$$

$$x_2 = T$$

$$x_3 = T$$



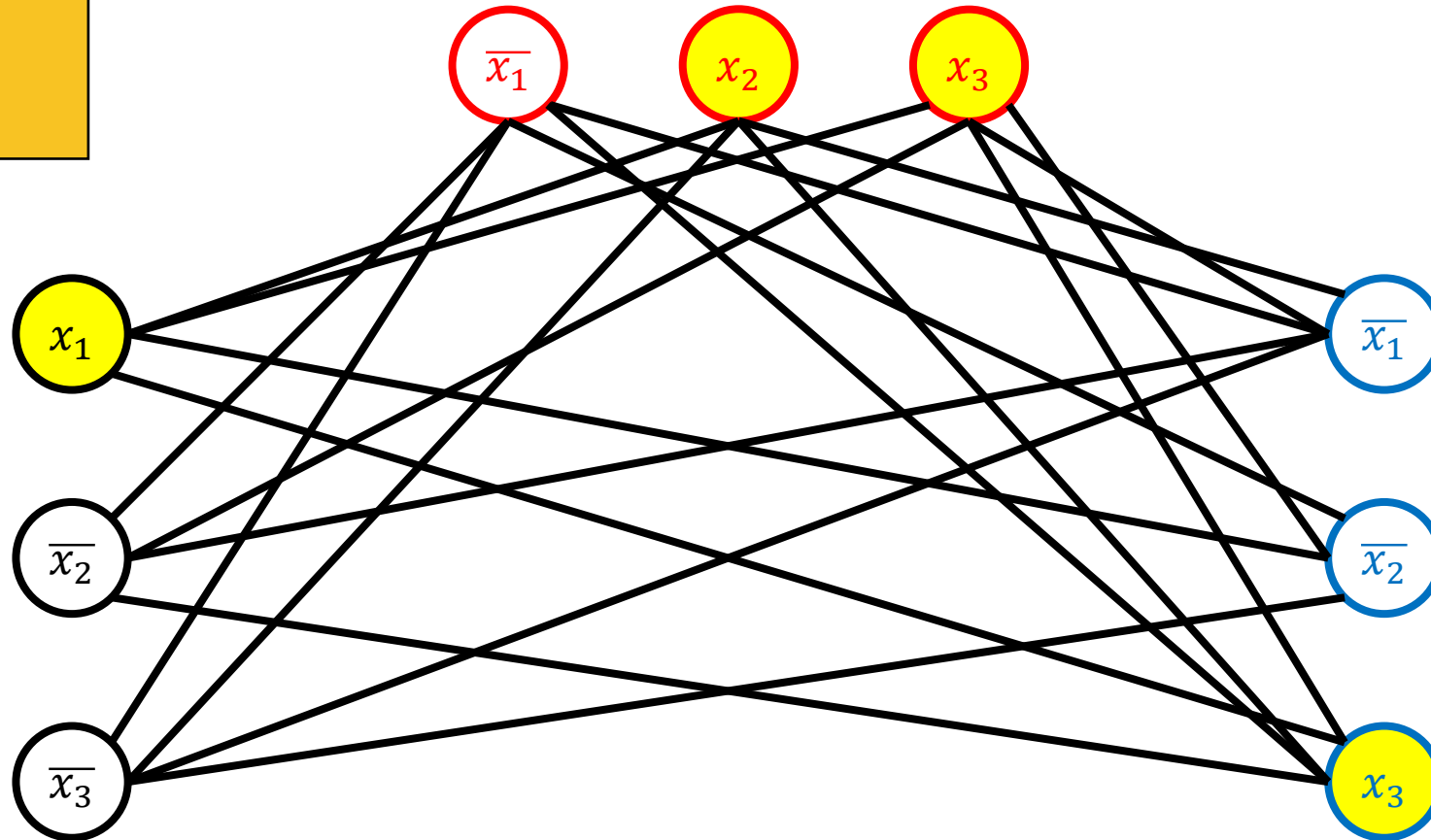
$$\phi = (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$

Consider the assignment:

$$x_1 = T$$

$$x_2 = T$$

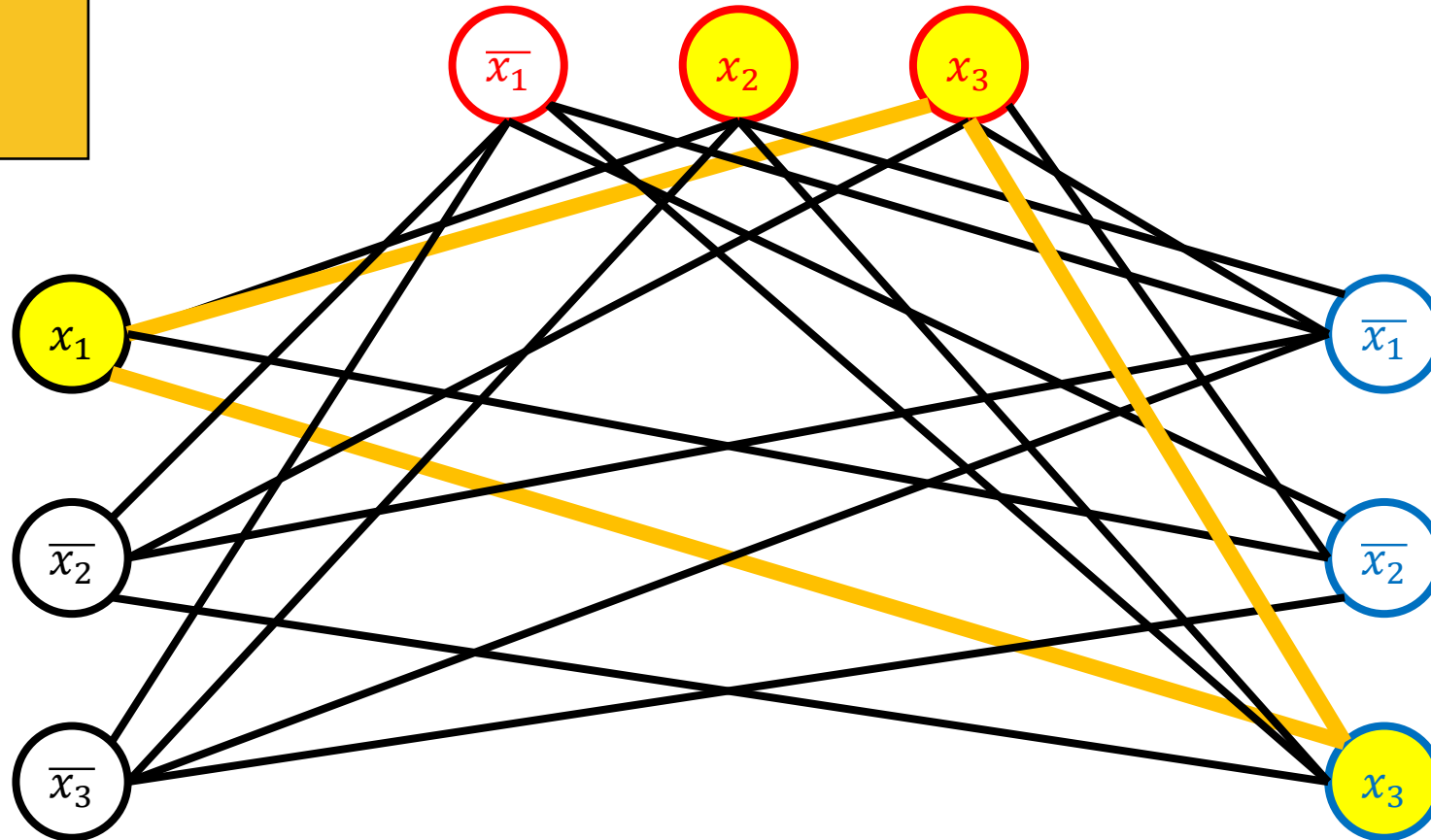
$$x_3 = T$$



$$\phi = (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$

Consider the assignment:

$$\begin{aligned} x_1 &= T \\ x_2 &= T \\ x_3 &= T \end{aligned}$$



The graph has a 3-clique!

NP and Reducibility

- We just proved that $3SAT \leq_P CLIQUE$.
- This means that if $CLIQUE \in P$, $3SAT \in P$!
 - (Spoiler alert: As far as we know, no, it isn't.)
- If we ever found a polynomial-time solution to a problem in NP, any other problem in NP that was reducible to that one would also be solvable in polynomial time.
- For some problems, this would be interesting. For some, it would change computer science completely.

NP-Completeness

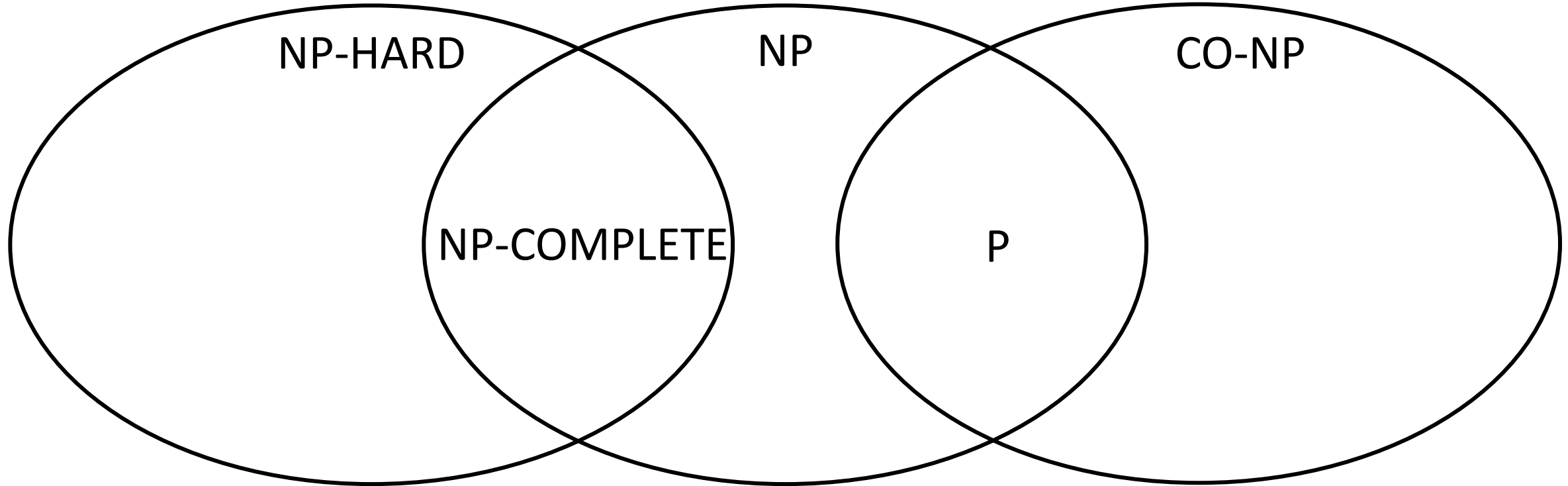
- We just proved that $3SAT \leq_P CLIQUE$.
- This means that if $CLIQUE \in P$, $3SAT \in P$!
 - (Spoiler alert: As far as we know, no, it isn't.)
- If we ever found a polynomial-time solution to a problem in NP, any other problem in NP that was reducible to that one would also be solvable in polynomial time.
- For some problems, this would be interesting. For some, it would change computer science completely.
- We say that a language B is **NP-complete** if:
 - $B \in NP$.
 - For every language $A \in NP$, $A \leq_P B$.
- This leads to several observations immediately. The two most important are:
 - If B is NP-complete, $C \in NP$, and $B \leq_P C$, then C is NP-complete.
 - **If B is NP-complete and $B \in P$, then $P = NP$.**
- The second has obviously never been invoked. The first is what we use to find most NP-complete languages. But before we can do that, we have to find our first one.

NP-Completeness

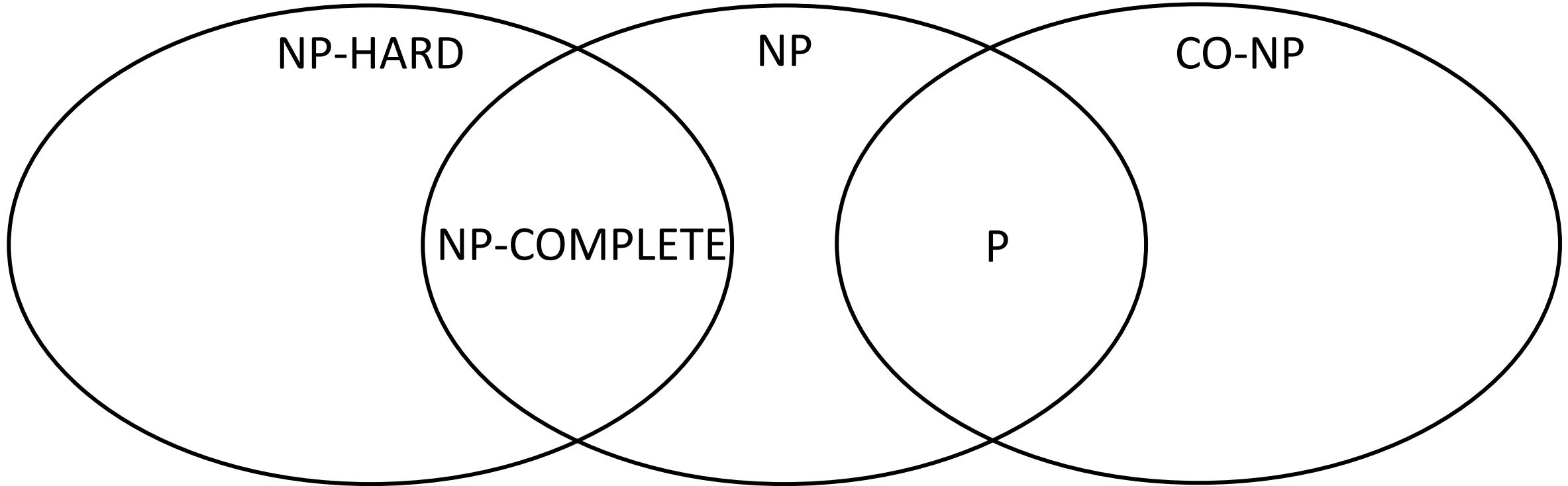
**The second
bullet point is
also known as
NP-Hard!**

- We say that a language B is **NP-complete** if:
 - $B \in \text{NP}$.
 - For every language $A \in \text{NP}$, $A \leq_P B$.
- This leads to several observations immediately. The two most important are:
 - If B is NP-complete, $C \in \text{NP}$, and $B \leq_P C$, then C is NP-complete.
 - If B is NP-complete and $B \in \text{P}$, then $\text{P} = \text{NP}$.
- The second has obviously never been invoked. The first is what we use to find most NP-complete languages. But before we can do that, we have to find our first one.

The Complexity Class Picture That We Know At Least...



The Complexity Class Picture That We Know At Least...



There's More Classes BTW...



Acknowledgement

Some Notes and content come from Dr. Gerber, Dr. Hughes, and Mr. Guha's COT4210 class and the Sipser Textbook, *Introduction to the Theory of Computation*, 3rd ed., 2013